

syntax	constructor	function
type usage	no need	need
write structure	explicit class (type paramter){ }	void function (type parameter){ }
{ }	: obj (parameter) initializer list	obj=parameter
Example	<pre>class EA{ public: explicit EA(int a) : b(a) { } private: Int b; };</pre>	<pre>class EA{ public: void function1(int a){ b=a; } private: Int b; };</pre>
function calling	as object's building	call function in driver program
purpose	initialize objects	edit or manipulate objects
implicit conversion	may happen	wont happen
need explicit	as parameter's number	no
calling method	EA e(123);	EA e; A.function1(123);

Build Header

.h	<pre>#include <iostream> class EA{ public: // no parameter EA() :b(0),c(0){ } // 1 parameter explicit EA(int a) :b(a),c(0){ } // 2 parameter EA(int a, int t) :b(a),c(t){ } private: int b,c; }</pre>	<pre>#include <iostream> class EA{ public: // no parameter void function1() { b = 0; c = 0; } // 1 parameter void function2(int a) { b = a; c = 0; } // 2 parameter void function3(int a, int t) { b = a; c = t; } private: int b,c; }</pre>
.cpp(driver program)	<pre>#include "EA.h" using namespace std; Int main(){ EA e; EA e1(); EA e2(123); EA e3(123,246); }</pre>	<pre>#include "EA.h" using namespace std; Int main(){ EA A; A.function1(); A.function2(123); A.function3(123,246); }</pre>

Information hiding & interface separation: different number parameter usage		
.h (interface)	<pre>#include <iostream> class EA{ public: // no parameter EA() // 1 parameter explicit EA(int a) // 2 parameter EA(int a, int t) private: int b,c; }</pre>	<pre>#include <iostream> class EA{ public: //no parameter void function1(; //1 parameter void function2(int a); //2 parameter void function3(int a, int t); private: int b,c; }</pre>
.cpp (function prototypes)	<pre>#include "EA.h" using namespace std; EA::EA() :b(0),c(0){ } EA::EA(int a) :b(a),c(0){ } EA::EA(int a, int t) :b(a),c(t){ }</pre>	<pre>#include "EA.h" using namespace std; void EA::function1(){ b=0; c=0; } void EA::function2(int a){ b=a; c=0; } void EA::function3(int a, int t) { b=a; c=t; }</pre>
.cpp(driver program)	<pre>#include "EA.h" using namespace std; int main(){ EA e1(); EA e2(123); EA e3(123,246); }</pre>	<pre>#include "EA.h" using namespace std; int main(){ EA e; e.function1(); e.function2(123); e.function3(123,246); }</pre>

Information hiding & interface separation: cross-class calling	
.h (interface)	<pre> #include <iostream> class EA{ public: void function1(int a, int t); private: int b,c; } class EB{ public: void function2(); private: EA ea; int d; </pre>
.cpp (function prototypes)	<pre> #include "EA.h" using namespace std; void EA::function1(int a, int t){ b=a; c=t; } int EA::geta()const{ return a; } EB::EB(int f) :d(f){ } void EB::function2(){ cout<<ea.geta*d<<endl; </pre>
.cpp(driver program)	<pre> #include "EA.h" using namespace std; int main(){ EA ea; ea.function1(int w,int y); EB eb(int h); eb.function2(); </pre>

